FORMATO DE DESARROLLO E INTEGRACIONES

FT-SL-011-APIs
Link de recaudos Credi10

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, jueves 16 de febrero del 2024

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 4
1.2 Detallar funcionalidades 5
1.3 Observaciones y/o Comentarios..... 15
2. CASOS DE PRUEBAS UNITARIAS 16

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

1. DETALLE TECNICO

1.1 Descripción del Requerimiento:

1.1.1 Requerimiento

Disponer API para suministrar información al link de recaudos de credi10 y aplicar el pago respectivo.

Se han establecido múltiples requerimientos donde cada uno presenta funcionalidades y procesos específicos que permitan la generación de información para link de recaudos.

A continuación, se detalla los procesos importantes para cumplir con el requerimiento.

- Crear parámetros de configuración para las APIs de Credi10.
- Crear endpoint para consultar información del cliente y enviar al link de recaudos.
- Crear endpoint para procesar el pago del link de recaudos.
- Crear endpoint compensar: es la que genera el recibo de caja, envía correo con el ticket de pago.

1.2 Detallar componentes existentes

Para el desarrollo de los requerimientos se cuenta con 1 solución desarrollada en .Net Framework 4.8 llamada “Créditos”.

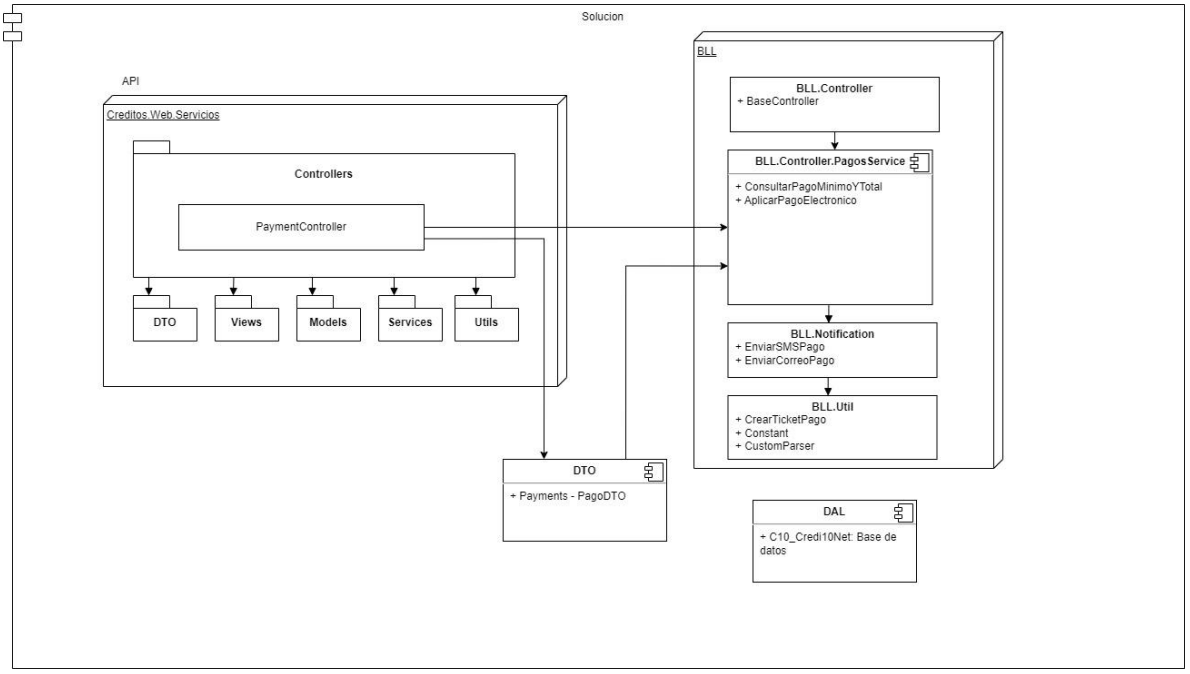
Se realiza creación del proyecto web API “*Creditos.Web.Servicios*” donde se establece el funcionamiento de los requerimientos correspondientes a Link de recaudo, botón ecommerce e integración B2C.

Nombre del Componente	Descripción
SOLUCION	
Creditos.Web.Servicios	PaymentController
Creditos.Web.Admin	Controladores y vistas implementadas para cumplir con el propósito de informar el valor del pago y recibir el valor que quieran aplicar.
Creditos.BLL	Proyecto que contiene las clases y métodos que implementan la lógica de negocio de la aplicación, esta biblioteca de clases será el encargado de calcular los pagos, procesarlos, generar y enviar el ticket. Este proyecto se integra con el proyecto Web.Admin, el proyecto DAL y el proyecto DTO.
Creditos.DAL	Proyecto encargado de tener el acceso y la interacción a la

	base de datos C10_Credi10NET de la solución.
Creditos.DTO	Proyecto que contiene los diferentes modelos usados en toda la aplicación.
Rotativa	Herramienta de línea de comandos para representar HTML en PDF, encargado de crear los tickets de impresión del pago.
BASE DE DATOS	
C10_Credi10NET	Tablas y Procedimientos almacenados

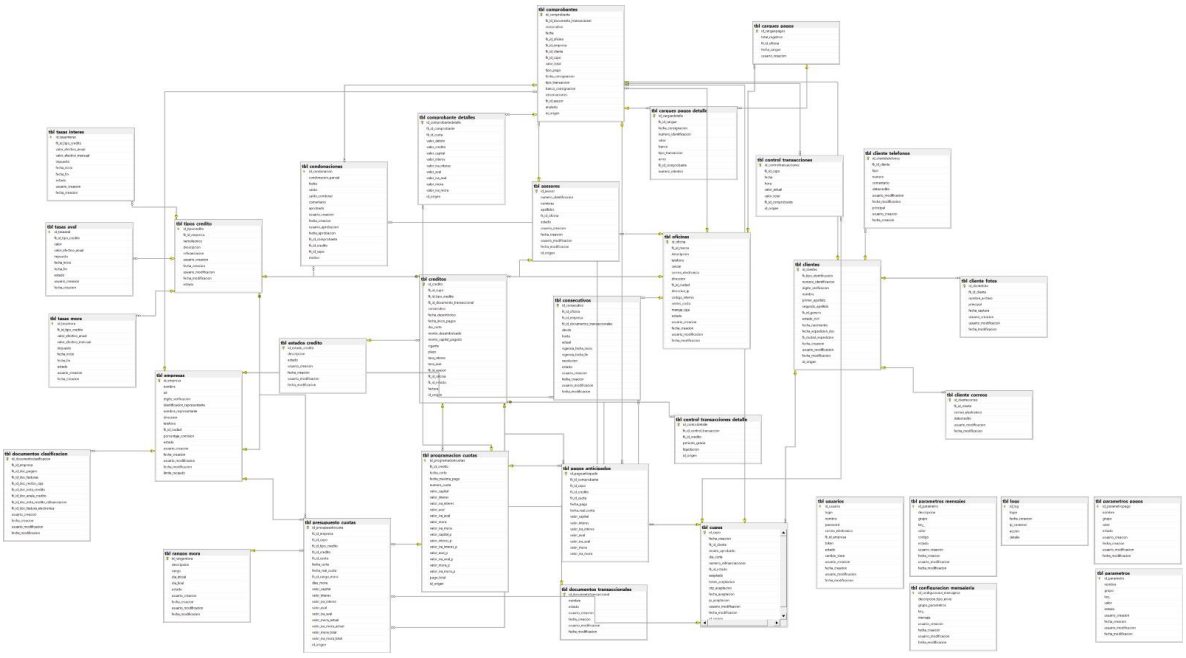
1.3 Detallar funcionalidades

1.3.1 Diagrama de Componentes



.

1.3.2 MER



Parametrizaciones

Descripción	Grupo	Valor
Mensaje mostrado cuando el pago se realiza correctamente	Pagos	Pago Realizado
Mensaje cuando hubo un problema al intentar generar el pago en tienda o consignación.	Pagos	Lo sentimos, error procesando el pago. Inténtalo de nuevo más tarde
Oficina para recibir pagos web (Sucursal)	Pagos	298
SMS - Mensaje Pago Realizado	SMS	Estimado cliente CREDI10 acabamos de recibir su pago por valor de ${0} con fecha de {1}.
Link conexión consumo API pagos	Link	https://apicredinet.studiof.com.co:32448/payment/paymentLink

1.3.3 Proceso

- **Operación “Consultar”**

Se implementa el componente PaymentController encargado de recibir por medio de la clase PaymentLink 2 tipos de peticiones diferenciadas por el atributo “operación”. Para este proceso se establece el valor “consultar” que tendrá la función de obtener la información de los datos del cliente, junto con los valores de su crédito: cedula, teléfono, correo electrónico, valor disponible, valor cupo, próxima cuota, día de corte, pago mínimo, pago total y por último un id liquidación que será el identificador del proceso que está iniciando el cliente con credi10.

Para esto se necesita recibir un objeto JSON en el cuerpo de la solicitud POST como el que se observa a continuación:

```
{
    "operacion": "consultar",
    "cliente": "StjTAmkYcr2zYkT69mkfZQ==",
    "usuario": "zunLY8QYl14=",
    "password": "63BlYoP7epw="
}
```

Cada uno de los atributos diferentes a la operación deberán ser enviados encriptados, estos serán la información del cliente y el usuario y la contraseña de quien podrá tener acceso a

consumir las APIs. (La creación del usuario y la contraseña deberá ser solicitada al equipo de TI).

```
0 referencias
public class PaymentController : Controller
{
    0 referencias
    public async Task<ActionResult> PaymentLink(PaymentRequestDto paymentRequestDto)
    {
        PagosService pagos = new PagosService();
        Encryption encription = new Encryption();
        var result = new OperationResponse();
        UserServices userServices = new UserServices();

        object dataObj = new object();
        object response = new object();

        if (paymentRequestDto.Operacion == "consultar")...
        else if (paymentRequestDto.Operacion == "compensar")...
        else
        {
            response = new
            {
                status = "error",
                data = dataObj,
                code = 405,
                mensaje = "Opción no disponible"
            };
        }

        return Json(response);
    }
}
```

Al realizar cualquier solicitud a la API de pagos entrará al método asíncrono llamado PaymentLink que será el encargado de procesar todas las solicitudes y dependiendo de la operación recibida en el cuerpo de la petición hacer una u otra tarea.

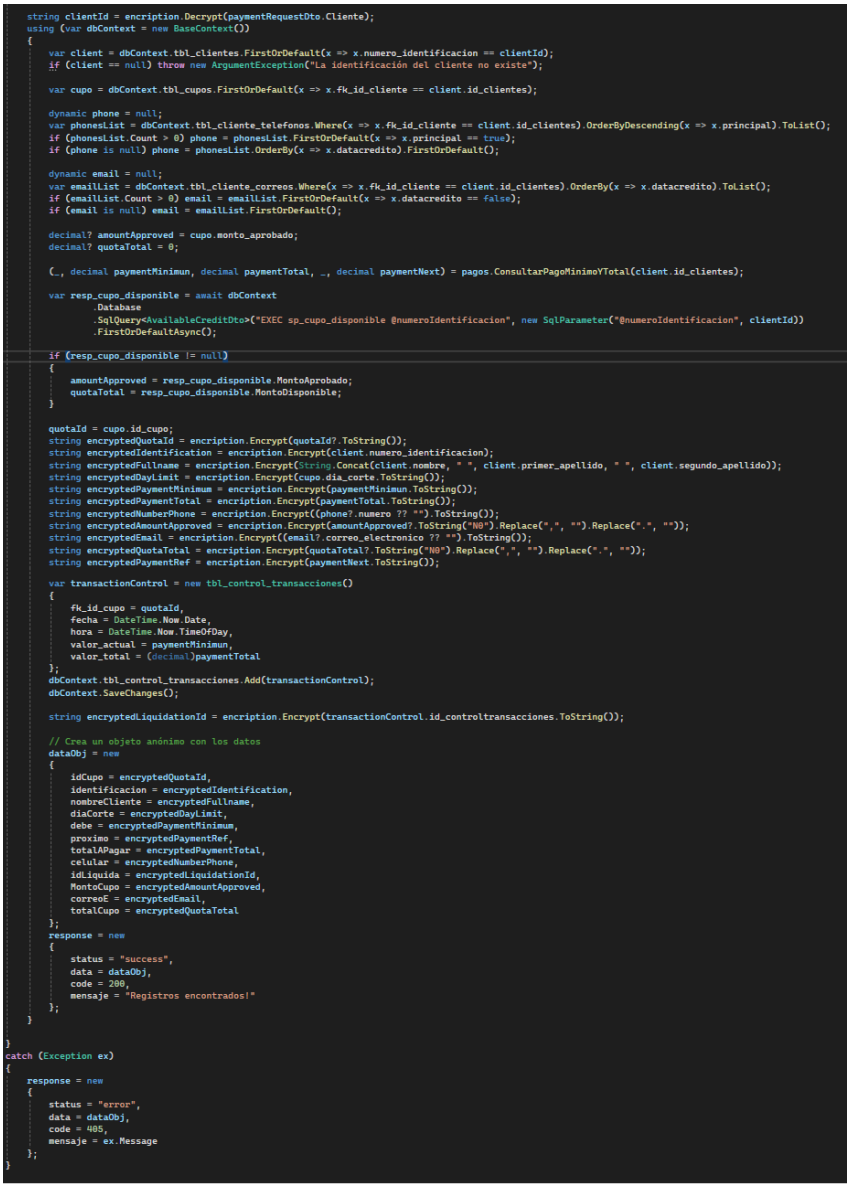
```
long? quotaId = null;

string username = encription.Decrypt(paymentRequestDto.Usuario);
string password = encription.Decrypt(paymentRequestDto.Password);
bool userChecked = userServices.ValidateUser(username, password);
if (!userChecked)
{
    response = new
    {
        status = "error",
        data = dataObj,
        code = 405,
        mensaje = "No autorizado"
    };

    return Json(response);
}
```

El objeto json recibido en la petición se desencripta con claves específicas y se inicia con el proceso validación del usuario y password suministrados, si no son correctas las

credenciales el sistema retornará “No Autorizado”, en caso de ser correctas, se obtendrá la información necesaria para devolver el objeto especificado en la API.



```
    string clientId = encription.Decrypt(paymentRequestDto.Cliente);
    using (var dbContext = new BaseContext())
    {
        var client = dbContext.tbl_clientes.FirstOrDefault(x => x.numero_identificacion == clientId);
        if (client == null) throw new ArgumentException("La identificación del cliente no existe");

        var cupo = dbContext.tbl_cupos.FirstOrDefault(x => x.fk_id_cliente == client.id_clientes);

        dynamic phone = null;
        var phonesList = dbContext.tbl_cliente_telefonos.Where(x => x.fk_id_cliente == client.id_clientes).OrderByDescending(x => x.principal).ToList();
        if (phonesList.Count > 0) phone = phonesList.FirstOrDefault(x => x.principal == true);
        if (phone is null) phone = phonesList.OrderBy(x => x.datacredito).FirstOrDefault();

        dynamic email = null;
        var emailList = dbContext.tbl_cliente_correos.Where(x => x.fk_id_cliente == client.id_clientes).OrderBy(x => x.datacredito).ToList();
        if (emailList.Count > 0) email = emailList.FirstOrDefault(x => x.datacredito == false);
        if (email is null) email = emailList.FirstOrDefault();

        decimal? amountApproved = cupo.monto_aprobado;
        decimal? quotaTotal = 0;

        (_, decimal paymentMinimun, decimal paymentTotal, _, decimal paymentNext) = pagos.ConsultarPagoMinimoYTotal(client.id_clientes);

        var resp_cupo_disponible = await dbContext
                .Database
                .SqlQuery<AvailableCreditDto>("EXEC sp_cupo_disponible @numeroIdentificacion", new SqlParameter("@numeroIdentificacion", clientId))
                .FirstOrDefaultAsync();

        if (resp_cupo_disponible != null)
        {
            amountApproved = resp_cupo_disponible.MontoAprobado;
            quotaTotal = resp_cupo_disponible.MontoDisponible;
        }

        quotaId = cupo.id_cupo;
        string encryptedQuotaId = encription.Encrypt(quotaId?.ToString());
        string encryptedIdentification = encription.Encrypt(client.numero_identificacion);
        string encryptedFullname = encription.Encrypt(String.Concat(client.nombre, " ", client.primer_apellido, " ", client.segundo_apellido));
        string encryptedDayLimit = encription.Encrypt(cupo.dia_corte.ToString());
        string encryptedPaymentMinimun = encription.Encrypt(paymentMinimun.ToString());
        string encryptedPaymentTotal = encription.Encrypt(paymentTotal.ToString());
        string encryptedNumberPhone = encription.Encrypt((phone?.numero ?? "").ToString());
        string encryptedAmountApproved = encription.Encrypt(amountApproved?.ToString("N0").Replace(",", "").Replace(".", ""));
        string encryptedEmail = encription.Encrypt((email?.correo_electronico ?? "").ToString());
        string encryptedQuotaTotal = encription.Encrypt(quotaTotal?.ToString("N0").Replace(",", "").Replace(".", ""));
        string encryptedPaymentRef = encription.Encrypt(paymentNext.ToString());

        var transactionControl = new tbl_control_transacciones()
        {
            fk_id_cupo = quotaId,
            fecha = DateTime.Now.Date,
            hora = DateTime.Now.TimeOfDay,
            valor_actual = paymentMinimun,
            valor_total = (decimal)paymentTotal
        };
        dbContext.tbl_control_transacciones.Add(transactionControl);
        dbContext.SaveChanges();

        string encryptedLiquidationId = encription.Encrypt(transactionControl.id_controltransacciones.ToString());

        // Crea un objeto anónimo con los datos
        dataObj = new
        {
            idCupo = encryptedQuotaId,
            identificacion = encryptedIdentification,
            nombreCliente = encryptedFullname,
            diaCorte = encryptedDayLimit,
            debe = encryptedPaymentMinimun,
            proximo = encryptedPaymentRef,
            totalAPagar = encryptedPaymentTotal,
            celular = encryptedNumberPhone,
            idLiquida = encryptedLiquidationId,
            MontoCupo = encryptedAmountApproved,
            correoE = encryptedEmail,
            totalCupo = encryptedQuotaTotal
        };
        response = new
        {
            status = "success",
            data = dataObj,
            code = 200,
            mensaje = "Registros encontrados!"
        };
    }
}
catch (Exception ex)
{
    response = new
    {
        status = "error",
        data = dataObj,
        code = 405,
        mensaje = ex.Message
    };
}
```

La operación consultar tendrá 2 objetivos, el primero es obtener la información del pago y el cupo del cliente que se recibió en la solicitud después de ser desencriptado y el segundo es crear un control de transacción con un identificador único del proceso que se está realizando.

Cumplido los dos objetivos se crea un objeto que tendrá como información el idCupo, el número de identificación, el nombre del cliente, su día de corte, cual es el pago mínimo y total a la fecha y el valor del próximo pago, el celular y correo, el monto disponible y aprobado y por último el id de la transacción creada. Todos los valores serán encriptados antes de ser enviados.

- **Operación Compensar**

Para este proceso se establece el valor “compensar” que tendrá la función de aplicar el pago en el sistema credi10 y notificar al cliente la confirmación de este.

Para esto se necesita recibir un objeto JSON en el cuerpo de la solicitud POST como el que se observa a continuación:


```
{
    "Operacion": "compensar",
    "IdLiquida": "gCYe4xUMr8w=",
    "IdCupo": "XSeQGxb0rDXWKsGtOptoZQ==",
    "ValorPagado": "LU4BKUrOsbI=",
    "Cliente": "o62LqIVP7lKK1KSpV5xPxA==",
    "CodSucursal": "gmoOkcxp2T0=",
    "Usuario": "d28IpPCRE7sQWYWcaOTcDw==",
    "Password": "v4aIAR2AgDfL17wQ6Pyr2g=="
}
```


Cada uno de los atributos diferentes a la operación deberán ser enviados encriptados, estos serán la información del cliente, el valor del pago que se realizó y se debe aplicar, el id de la transacción con la que se realizó el proceso junto a la oficina que en este caso es la sucursal de Pagos Web y por último el usuario y la contraseña de quien podrá tener acceso a consumir las APIs. (La creación del usuario y la contraseña deberá ser solicitada al equipo de TI).

El objeto json recibido en la petición se desencripta con claves específicas y se inicia con el proceso validación del usuario y password suministrados, si no son correctas las credenciales el sistema retornará “No Autorizado”, en caso de ser correctas, se procedera con la aplicación del pago.

```
else if (paymentRequestDto.Operacion == "compensar")
{
    try
    {
        string clientId = encription.Decrypt(paymentRequestDto.Cliente);
        string valuePaid = paymentRequestDto.ValorPagado;
        string balanceId = encription.Decrypt(paymentRequestDto.IdLiquida);

        using (var dbContext = new BaseContext())
        {
            var client = dbContext.tbl_clientes.FirstOrDefault(x => x.numero_identificacion == clientId);
            if (client == null) throw new ArgumentException("La identificación del cliente no existe");

            dynamic phone = null;
            var phonesList = dbContext.tbl_cliente_telefonos.Where(x => x.fk_id_cliente == client.id_clientes).OrderByDescending(x => x.principal).ToList();
            if (phonesList.Count > 0) phone = phonesList.FirstOrDefault(x => x.datacredito == false);
            if (phone is null) phone = phonesList.FirstOrDefault();

            dynamic email = null;
            var emailList = dbContext.tbl_cliente_correos.Where(x => x.fk_id_cliente == client.id_clientes).ToList();
            if (emailList.Count > 0) email = emailList.FirstOrDefault(x => x.datacredito == false);
            if (email is null) email = emailList.FirstOrDefault();

            var sucursal = ConfigurationManager.AppSettings["Sucursal"];
            int idSucursal = int.Parse(sucursal);
            var oficina = dbContext.tbl_oficinas.FirstOrDefault(o => o.id_oficina == idSucursal);

            PagoDTO pagoDto = new PagoDTO();
            pagoDto.ValorPago = decimal.Parse(valuePaid);

            pagoDto.Cliente = String.Join(" ", client.nombre, client.primer_apellido, client.segundo_apellido);
            pagoDto.Cedula = clientId;
            pagoDto.Celular = (phone?.numero ?? "");
            pagoDto.Correo = (email?.correo_electronico ?? "");
            pagoDto.IdCliente = client.id_clientes;
            pagoDto.IdOficina = int.Parse(sucursal);
            pagoDto.Oficina = (oficina?.tbl_marcas?.nombre + " " + oficina?.descripcion) ?? "STF GROUP S.A.";
            pagoDto.Empresa = oficina?.tbl_marcas?.tbl_empresas?.nombre ?? "STF GROUP S.A.";
            pagoDto.IdEmpresa = oficina?.tbl_marcas?.tbl_empresas?.id_empresa ?? 1;
            pagoDto.Nit = oficina?.tbl_marcas?.tbl_empresas?.nit ?? "805003626";
            pagoDto.Date = DateTime.Now;
            pagoDto.ValorPago = decimal.Parse(valuePaid);
            pagoDto.TipoPago = "Pago Online";
            pagoDto.Asesor = "";
            pagoDto.Telefono = oficina?.telefono ?? "";
            pagoDto.Ciudad = oficina?.tbl_marcas?.tbl_empresas?.tbl_ciudades?.descripcion ?? "";
            pagoDto.Direccion = oficina?.direccion ?? "";

            result = pagos.AplicarPagoElectronico(pagoDto, long.Parse(balanceId), ControllerContext);

            if (result.IsError)
            {
                response = new
                {
                    status = "error",
                    data = dataObj,
                    code = 405,
                    mensaje = "No se generó número de recibo"
                };
            }
            else
            {
                response = new
                {
                    status = "success",
                    data = dataObj,
                    code = 200,
                    mensaje = "Valor compensado y recibo enviado!"
                };
            }
        }
    }
```

La operación compensar tendrá 2 objetivos que serán ejecutados en el método llamado *AplicarPagoElectronico()*, estos objetivos son la aplicación del pago recibido y la notificación de este al cliente con la información de este.

Cuando el proceso termine de ejecutarse se construirá un objeto anónimo con la información del resultado de la operación que será retornada como respuesta de manera encriptada.

Otros procesos

A. Validar el usuario

```
public bool ValidateUser(string username, string password)
{
    var pass = "";

    try
    {
        using (var dbContext = new BaseContext())
        {
            var result = dbContext.tbl_usuarios.FirstOrDefault(u => u.estado && u.login.ToUpper() == username.ToUpper());
            if (result != null)
            {
                if (result.cambio_clave != null) pass = SecurityUtil.Sha256(password);
                else pass = SecurityUtil.Md5(password);

                return result.password == pass;

            }
            return false;
        }
    }
    catch(Exception ex)
    {
        return false;
    }
}
```

Método que valida si el usuario y la contraseña enviada en la petición después de ser descifrada están registrados como usuarios en el sistema para poder acceder a la información en credi10.

B. Encriptar

```
public string Encrypt(string valor)
{
    string svalor = string.Empty;

    if (!string.IsNullOrEmpty(valor))
    {
        try
        {
            using (var md5 = new MD5CryptoServiceProvider())
            {
                using (var tdes = new TripleDESCryptoServiceProvider())
                {
                    tdes.Key = md5.ComputeHash(Encoding.UTF8.GetBytes(SKeyWord));
                    tdes.Mode = CipherMode.ECB;
                    tdes.Padding = PaddingMode.PKCS7;

                    using (var transform = tdes.CreateEncryptor())
                    {
                        byte[] textBytes = Encoding.UTF8.GetBytes(valor);
                        byte[] bytes = transform.TransformFinalBlock(textBytes, 0, textBytes.Length);
                        svalor = Convert.ToBase64String(bytes, 0, bytes.Length);
                    }
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error " + ex.Message);
            return svalor;
        }
    }
    return svalor;
}
```

Método usado para encriptar información con TripleDES (Triple Data Encryption Standard).

C. Desencriptar

```
public string Decrypt(string valor)
{
    string svalor = string.Empty;

    if (!string.IsNullOrEmpty(valor))
    {
        try
        {
            using (var md5 = new MD5CryptoServiceProvider())
            {
                using (var tdes = new TripleDESCryptoServiceProvider())
                {
                    tdes.Key = md5.ComputeHash(Encoding.UTF8.GetBytes(SKeyWord));
                    tdes.Mode = CipherMode.ECB;
                    tdes.Padding = PaddingMode.PKCS7;

                    using (var transform = tdes.CreateDecryptor())
                    {
                        byte[] cipherBytes = Convert.FromBase64String(valor);
                        byte[] bytes = transform.TransformFinalBlock(cipherBytes, 0, cipherBytes.Length);
                        svalor = Encoding.UTF8.GetString(bytes);
                    }
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error " + ex.Message);
            return svalor;
        }
    }
    return svalor;
}
```

Método usado para desencriptar una cadena de string que este encriptado con TripleDES (Triple Data Encryption Standard).

D. Consultar pago minimo y total

Método que retorna el pago minimo, el pago total y el próximo pago que tendrá un cliente en credi10 y serán los valores que el usuario podrá ver en pantalla para continuar con el proceso por el link de recaudo. Este método es explicado en el documento Documentacion GenerarPagos.docx

E. Aplicar Pago Electrónico

Método que aplicara el pago usando la misma lógica contenida en el documento **GenerarPagos.docx** pero que en vez de crear una transacción en el proceso recupera la transacción ya realizada en la operación “consultar”. Manteniendo la lógica usada para asentar y crear el comprobante de pago.



```
1 referencia
public OperationResponse AplicarPagoElectronico(PagoDTO payment, long idTransaccion, ControllerContext context = null)
{
    var result = new OperationResponse();
    var ticket = new Ticket();
    var saldo = payment.ValorPago;
    var SeCreoDetalleTransaccion = false;

    var cupo = Connection.ContextC10.tbl_cupos.FirstOrDefault(c => c.fk_id_cliente == payment.IdCliente);
    var (fechaCorteMesActual, fechaPagoMesActual, fechaUltimoCorte) = ConsultarFechaCorteCliente((int)cupo.dia_corte);
    var creditos = Connection.ContextC10.tbl_creditos.Where(c => c.vigente && c.fk_id_cupo == cupo.id_cupo).OrderBy(c => c.fecha_desembolso).ToList();

    ticket.fechaUltimoCorte = fechaUltimoCorte;
    ticket.fechaProximoCorte = fechaCorteMesActual;
    ticket.fecha = DateTime.Now;
    var asesor = Connection.ContextC10.tbl_asesores.FirstOrDefault(a => a.numero_identificacion.Equals(payment.Asesor));

    ticket.empresa = ParameterManager.GetParam(Constant.ParamKey.EmpresaTicket).Value;
    ticket.nit = ParameterManager.GetParam(Constant.ParamKey.NitTicket).Value;
    ticket.direccion = payment.Direccion;
    ticket.ciudad = payment.Ciudad;
    ticket.telefono = payment.Telefono;
    ticket.nombreCliente = payment.Cliente;
    ticket.oficina = payment.Oficina;
    ticket.cedula = payment.Cedula;
    ticket.valorPago = payment.ValorPago;
    ticket.detallePago = new List<DetalleTicket>();

    //creamos el ámbito de la transacción
    using (var dbContextTransaction = Connection.ContextC10.Database.BeginTransaction())
    {
        try
        {
            var documentoClasificacion = Connection.ContextC10.tbl_documentos_clasificacion.First(d => d.fk_id_empresa == payment.IdEmpresa);
            var consecutivo = new tbl_consecutivos();
            var documento = "";

            if (payment.TipoPago.Equals(Constant.TipoPagoComprobante.Refinaciacion))
            {
                consecutivo = Connection.ContextC10.tbl_consecutivos.FirstOrDefault(c => (c.fk_id_oficina == payment.IdOficina)
                && c.fk_id_documentos_transaccionales == documentoClasificacion.fk_id_doc_nota_credito_refinanciacion
                && c.vigencia_fecha_inicio <= DateTime.Today && c.vigencia_fecha_fin >= DateTime.Today);
                documento = documentoClasificacion.fk_id_doc_nota_credito_refinanciacion;
            }
            else
            {
                consecutivo = Connection.ContextC10.tbl_consecutivos.FirstOrDefault(c => (c.fk_id_oficina == payment.IdOficina)
                && c.fk_id_documentos_transaccionales == documentoClasificacion.fk_id_doc_recibo_caja
                && c.vigencia_fecha_inicio <= DateTime.Today && c.vigencia_fecha_fin >= DateTime.Today);
                documento = documentoClasificacion.fk_id_doc_recibo_caja;
            }

            //Ya no se crea si no que se recupera, si este ya tiene un comprobante asociado no se trae la transaccion y se crea una excepcion controlada
            //para controlar error evidenciado en el link de recaudo con una doble transaccion creada por parte de ellos con el mismo idLiquida
            var transaccion = Connection.ContextC10.tbl_control_transacciones.FirstOrDefault(t => t.id_controltransacciones == idTransaccion && t.fk_id_
            if (transaccion == null)
            {
                throw new CustomException("Error Duplicidad del pago");
            }

            var comprobante = new tbl_comprobantes()
            {
                fk_id_documento_transaccional = documento,
                consecutivo = consecutivo.actual + 1,
                fecha = DateTime.Now,
                fk_id_oficina = payment.IdOficina,
                fk_id_empresa = payment.IdEmpresa,
                fk_id_cliente = payment.IdCliente,
                fk_id_cupo = cupo.id_cupo,
                valor_total = payment.ValorPago,
                tipo_pago = payment.TipoPago,
                fecha_consignacion = payment?.FechaConsignacion,
                tipo_transaccion = payment?.TipoTransaccion,
                banco_consignacion = payment?.Banco,
                observaciones = payment?.Observaciones,
                fk_id_asesor = asesor?.id_asesor,
                anulado = false
            };
            Connection.ContextC10.tbl_comprobantes.Add(comprobante);

            //Asigno el comprobante que se creara a la transaccion
            transaccion.fk_id_comprobante = comprobante.id_comprobante;

            //SUMAR AL CONSECUTIVO
            consecutivo.actual += 1;

            ticket.consecutivo = comprobante.fk_id_documento_transaccional + comprobante.consecutivo.ToString();

            result = AcentuarPago(creditos, saldo, SeCreoDetalleTransaccion, transaccion, comprobante, cupo, ticket, transaccion.fecha, fechaCorteMesAct

            //Guardamos en la base de datos
            Connection.ContextC10.SaveChanges();
            //Hacemos commit de todos los datos si no hubo ningun error
            dbContextTransaction.Commit();

            //Si se aplica y guarda correctamente el pago realizado se actualiza el estado del credito despues de, NO SE PUEDE HACER ANTES EN EL CICLO
            foreach (tbl_creditos credito in creditos)
            {
                //Devuelve true si hubo un cambio en el estado del credito. Por si se quiere guardar este dato en un futuro
                var a = Connection.ContextC10.sp_actualizacion_vigencia_credito(credito.id_credito).First();
            }

            //Se hace la validacion por si el pago hecho actualmente activa el cupo automaticamente, es un proceso interno no se necesita respuesta.
            Connection.ContextC10.sp_disminuir_y_activar_cupo(cupo.id_cupo);

            result.Data = comprobante.id_comprobante.ToString();
            result.IsError = false;
            result.Status = true;
            result.Completed = true;
            result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.PagoExitoso).Value;
        }
        catch (Exception ex)
        {
            //hacemos rollback si hay excepción
            dbContextTransaction.Rollback();
            result.IsError = true;
            result.Status = false;
            result.Completed = false;
            result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.ErrorProcesoPago).Value;
        }
    }
```

1.3.4 Documentación de Arquitectura

Lógica completa para generar el pago en credi10 **GenerarPagos.docx.**

Demas procesos del API de servicios:

- BotonEcommerce.docx
- B2C.docx

1.3.5 Log

En este proceso no es generado ningún log de auditoría.

1.3.6 Lista de Programas, archivos, paquetes

- **sp_cupo_disponible:**
  Procedimiento almacenado que retorna el valor del cupo aprobado y el monto disponible que tiene en credi10 de un cliente especifico.

1.3.7 Lista de modificaciones hacia la base de datos

Script 1. Creación de tablas: Para crear el esquema completo de la base de datos de la aplicación.

Creacion_BD_C10_Credi10Net.sql

1.4 Observaciones y/o Comentarios

En el archivo coleccionPostman.json se encontrará la colección que podrá ser exportada a postman con todas las consultas que podrán ser realizadas al API de servicios.

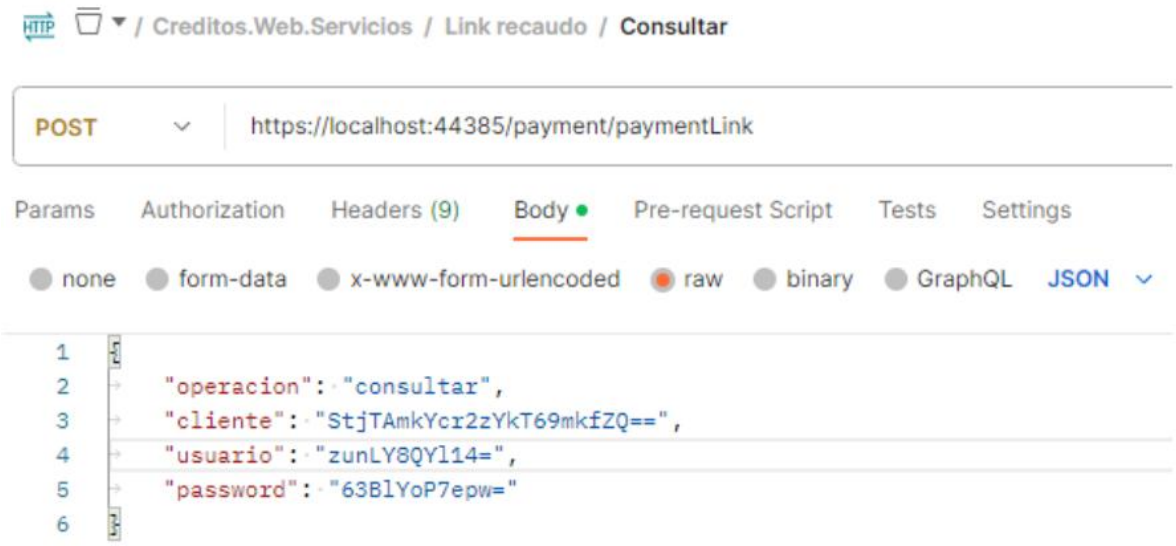
2. CASOS DE PRUEBAS UNITARIAS

A continuación, se demuestra las pruebas funcionales hechas en el ambiente de pruebas.

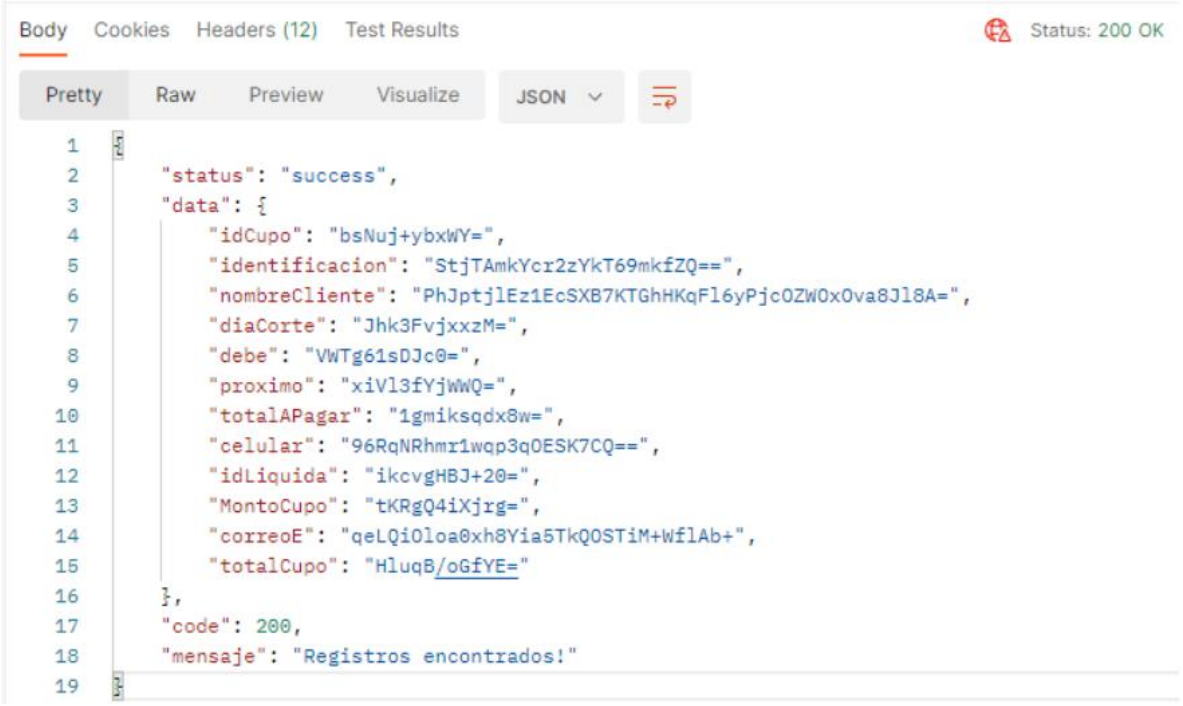
Para realización de pruebas unitarias de la API se hace uso de la herramienta Postman.

1. Para llevar a cabo las pruebas de generación de link de recaudo, se ejecuta el request en Postman y se obtiene la siguiente respuesta satisfactoria:

   - Envío de solicitud de generación de link de recaudo.

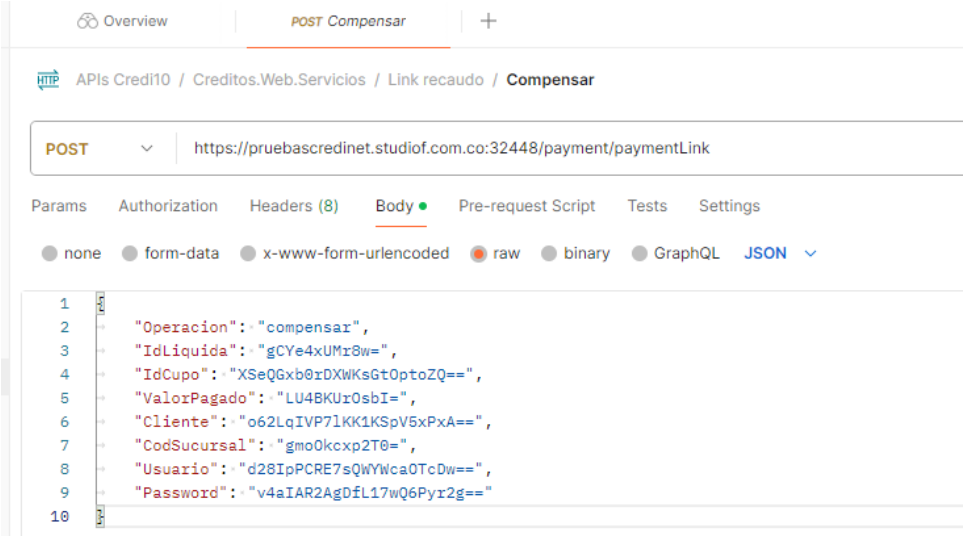


   - Respuesta a la solicitud de generación de link de recaudo



2. Para llevar a cabo las pruebas de registro de pago del link de recaudo, se ejecuta el request en Postman y se obtiene la siguiente respuesta satisfactoria:

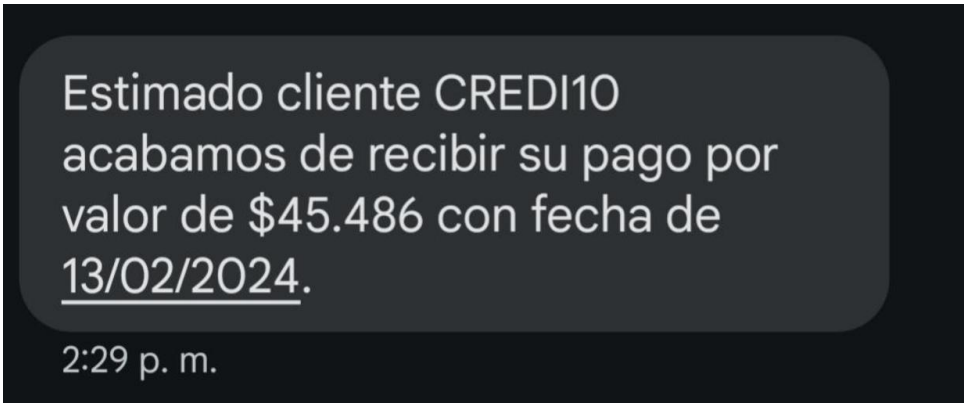
- Envío de registro de pago de link de recaudo



- Respuesta de registro de pago de link de recaudo



- Mensaje enviado al cliente por SMS para confirmar pago.



- Correo enviado al cliente para confirmar pago.



- Ticket de pago adjunto al correo.

Santiago de Cali Fecha: 13/02/2024 2:28 p. m.

STF GROUP S.A.

805003626

Sede Principal Credi10 Pagos Web
Cra. 34 #10 581, Yumbo

4851515

Datos del Recibo
Nro Id:
10
Nombre Cliente:
Jesus
Recibo de Caja
RC672092

Detalle del Pago

Valor Total
45.486

Cap.	Int.	Mora	Aval	Iva
44.950	450	0	0	86

Fecha Impresion: 13/02/2024 2:28:40 p. m.

ID	CASO DE PRUEBA	RESULTADO ESPERADO	APROBADO
1	Ejecución de API generación link de recaudo	Generación de información de link de recaudo con encriptación	SI
2	Ejecución de API registro link de recaudo	Confirmación de registro de pago realizado	SI